

Como gerar sua chave de API — 5 caminhos (3 free, 2 pagos com crédito)

O **default** e o **Caminho A (Gemini)** — free, suficiente pros 7 labs, e os notebooks **carregam a chave sozinhos** (Colab Secrets → `.env` → prompt). Os Caminhos B-E (Groq, Ollama, OpenAI, Anthropic) são **alternativas documentadas**: funcionam, mas exigem editar a 1ª célula do notebook (ver “Como o notebook escolhe o provider” adiante).

TL;DR — Decisão em 30 segundos



Caminho	Provider	Cartao?	Quota / credito free	Embeddings?	Quando usar
A (default)	Google Gemini	Nao	15 req/min, 1000 req/dia	nativo	Voce tem conta Google
B	Groq	Nao	30 req/min, 1000 req/dia	x usar local	Quer latencia ~10x menor
C	Ollama (local)	—	ilimitada (sua maquina)	local	Sem internet / privacidade
D	OpenAI	Sim*	US\$5 credito (expira 14d)	nativo	Medir custo USD no LAB-005
E	Anthropic	Sim*	US\$5 credito (expira 14d)	x usar local/OpenAI	Medir custo USD no LAB-005

*D/E pedem cartao apenas para verificacao; os US\$5 nao sao cobrados. Clame o credito **no dia da aula** (expira em 14 dias apos o “Claim”). **Embeddings:** Groq e Anthropic **nao tem** endpoint de embeddings — nos LAB-002/003/004 combine com `sentence-transformers` local (ja vem no `uv sync`) ou use embeddings da OpenAI/Gemini. Os 5 caminhos cobrem 100% dos labs. Escolha um e siga.

Como o notebook escolhe o provider

Caminho A (Gemini) — sem editar nada. A 1a célula de cada notebook carrega a `GEMINI_API_KEY` automaticamente, tentando nesta ordem: **Colab Secrets** (painel da chave) → `.env` local → **prompt** interativo (`getpass`). Basta a chave estar em um desses lugares.

Caminhos B-E (alternativas) — editar a 1a célula. Para usar Groq/Ollama/OpenAI/ Anthropic, substitua o `client` (e o modelo/embeddings) na 1a célula conforme a tabela abaixo. Coloque a chave no `.env` (ou no Secrets do Colab) com o nome correspondente.

Caminho	<code>base_url</code> do client	Modelo chat	Embeddings nos LAB-002/003/004
A Gemini	<code>https://generativelanguage.googleapis.com/v1beta/openai/</code>	<code>gemini-2.5-flash-lite</code>	<code>gemini-embedding-001</code> (nativo)
B Groq	<code>https://api.groq.com/openai/v1</code>	<code>llama-3.3-70b-versatile</code>	<code>sentence-transformers</code> local
C Ollama	<code>http://localhost:11434/v1</code>	<code>qwen3:8b</code>	<code>sentence-transformers</code> local
D OpenAI	(default do SDK)	<code>gpt-4o-mini</code>	<code>text-embedding-3-small</code> (nativo)
E Anthropic	<code>https://api.anthropic.com/v1/</code>	<code>claude-haiku-4-5</code>	<code>text-embedding-3-small</code> (OpenAI) ou local

Todos os 5 usam o **mesmo SDK** (`from openai import OpenAI`) — muda so a `base_url` e o nome do modelo. Por isso o código dos labs e quase identico entre caminhos.

Caminho A — Google Gemini (default da turma)

Por que e o default: maior contexto free do mercado (1M tokens), nao pede cartao, conta Google quase todo mundo tem, e o modelo `gemini-2.5-flash-lite` e suficiente pros 7 notebooks.

Passo a passo

1. Abra <https://aistudio.google.com/apikey> no navegador
2. **Faca login** com qualquer conta Google (gmail pessoal ou institucional)
3. Clique em **“Get API key”** (ou **“Criar chave de API”**, conforme o idioma)
4. Selecione **“Create API key in new project”** — aceita o projeto default que ele sugere
5. **Copie a chave** que aparece (comeca com `AIza...`, ~39 caracteres)
6. **Guarde com seguranca** — voce nao consegue ver de novo depois de fechar a aba; precisaria gerar outra

Demora ~30 segundos do login ate a chave estar no clipboard.

Quotas que voce tem (free tier, 2026)

Modelo	Req/min	Req/dia	Tokens/min	Pra que serve no curso
gemini-2.5-flash-lite	15	1000	250k	LLM principal (default dos notebooks)
gemini-2.5-flash	10	250	250k	RAGAS judge no LAB-003
gemini-2.5-pro	5	100	250k	papel “premium” no LAB-005
gemini-embedding-001	—	—	250k	embeddings nos LAB-002/003/004

Em dezembro/2025 o Google cortou rate limits free em 50–80%. Os limites acima são pós-corte. São suficientes pros 18h se você **nao** disparar 100 chamadas em 1 minuto. Se der erro `429 Too Many Requests`, aguarde 60s.

Adicionar a chave — Jupyter local OU Google Colab

Jupyter local: crie o `.env` na raiz da pasta descompactada (`software-com-ia-generativa/`):

```
echo 'GEMINI_API_KEY=AIZA...sua-chave-aqui' > .env
```

Google Colab: não use `.env`. Abra o painel de **Secrets** (ícone de **chave** na barra lateral esquerda) → **Add new secret** → nome `GEMINI_API_KEY`, valor a chave, e ative **Notebook access**. A 1ª célula do notebook lê esse Secret automaticamente.

Substitua `AIZA...sua-chave-aqui` pelo valor real. O Secret do Colab **nao** fica salvo no `.ipynb` (seguro pra compartilhar o notebook depois).

Caminho B — Groq (rapido, sem conta Google)

Por que escolher: se você ainda não tem conta Google ou prefere latência muito baixa (Groq roda em LPUs próprias e responde em ~200ms — ~10x mais rápido que Gemini Flash). Ótimo pro LAB-001 (agent CLI com loop de tool-use, que dispara várias chamadas seguidas).

Passo a passo

1. Abra <https://console.groq.com>
2. Clique em **“Sign up”** — pode usar email/senha, Google, ou GitHub
3. Após confirmar email, vá em **API Keys** no menu lateral esquerdo
4. Clique em **“Create API Key”**
5. De um nome (ex: `aulas-ppi-mod4`) e clique **Submit**
6. **Copie a chave** (começa com `gsk_...`) — ela só aparece **uma vez**

Quotas que você tem (free tier, 2026)

Modelo	Req/min	Req/dia	Tokens/min	Equivalente Gemini
llama-3.3-70b-versatile	30	1000	6k	similar a gemini-2.5-flash
qwen-qwq-32b	30	1000	6k	otimo em CoT
deepseek-r1-distill-llama-70b	30	1000	6k	bom raciocinio

Atencao: Groq **nao tem embeddings**. Se for usar Groq, combine com `sentence-transformers` local pros LAB-002/003/004 (ja vem no `uv sync`).

Adicionar a chave no `.env`

```
echo 'GROQ_API_KEY=gsk_...sua-chave-aqui' > .env
```

Caminho C — Ollama (100% local, sem chave nenhuma)

Por que escolher: privacidade total (nada sai da sua maquina), sem rate limit, funciona **offline**. Bom se a WiFi da aula travar, se sua conta Gemini esgotar a quota, ou se voce quer entender o trade-off cloud vs local.

Pre-requisito: laptop com pelo menos **8 GB de RAM** disponiveis para o modelo (16 GB total e o ideal). Sem isso, vai swappar e ficar lento.

Passo a passo

1. **Instale o Ollama:**

```
curl -fsSL https://ollama.com/install.sh | sh
```

No macOS, tambem da pra baixar `.dmg` em <https://ollama.com/download>. No Windows: instalador oficial WSL2 (idem link acima).

2. **Inicie o servico** (ja deve estar rodando apos a instalacao):

```
ollama serve
```

Se ja estiver ativo, vai dar erro `address already in use` — pode ignorar.

3. **Baixe o modelo** (~5 GB, baixe **antes da aula** com WiFi decente):

```
ollama pull qwen3:8b
```

4. **Teste:**

```
ollama run qwen3:8b "Diga apenas: ping"
```

Deve imprimir `ping` em ~2 segundos.

Modelos recomendados

Modelo	RAM minima	Tool-use	Recomendado para
qwen3:8b	8 GB	estavel	default do Caminho C
qwen2.5:7b	8 GB	estavel	fallback
gemma3:9b	12 GB	nativo	se voce tem 16 GB+
llama3.3:70b	48 GB	bom	so com GPU dedicada

Em 2026 a familia **Qwen** e a mais estavel em function-calling local. Evite modelos pequenos demais (<7b) — falham em CoT longa.

Configurar para os notebooks

Como o Ollama nao tem chave de API, voce nao escreve nada de `_API_KEY` no `.env`. Apenas avise o cliente OpenAI que a base URL e local:

```
# .env para o Caminho C
OPENAI_API_KEY=ollama-local
OPENAI_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen3:8b
```

Limitacoes do Caminho C

- Qualidade **menor** que Gemini Flash em CoT longa (LAB-003 RAGAS pode dar scores mais baixos que o esperado — e esperado para Tier 3)
- Embeddings: usar `sentence-transformers` local (ja esta nas deps do `uv sync`)
- RAGAS roda ~3x mais lento (LLM-as-judge fica em qwen local)

Caminho D — OpenAI (pago, US\$5 de credito free)

Por que escolher: e o caminho que melhor serve o **LAB-005** (semantic cache + cheap-first routing), porque o ponto pedagogico la e medir **custo em USD por requisicao** — so faz sentido com price card real. Tem embeddings nativos, entao nao precisa de fallback nos LAB-002/003/004.

Passo a passo

- Abra <https://platform.openai.com> e crie conta
- Va em **Settings > Billing** e adicione um cartao (apenas verificacao; o credito free de US\$5 cobre este curso ~3x)
- Va em **API keys > Create new secret key**
- Copie a chave** (comeca com `sk-...`) — so aparece uma vez

Modelos e pricing (USD por 1M tokens, conferir antes da aula)

Modelo	Papel	Input	Output	Embeddings
gpt-4o-mini	cheap (default)	0.15	0.60	—
gpt-4o	premium (LAB-005)	2.50	10.00	—
text-embedding-3-small	embeddings	0.02	—	

Adicionar a chave no `.env`

```
echo 'OPENAI_API_KEY=sk-...sua-chave-aqui' > .env
```

O credito free **expira em 14 dias** apos clicar em “Claim” — clame **no dia da aula**.

Caminho E — Anthropic (pago, US\$5 de credito free)

Por que escolher: alternativa ao Caminho D para o LAB-005 (custo USD real), com os modelos Claude. Usa o **endpoint OpenAI-compatible** da Anthropic, entao o mesmo SDK `openai` funciona — so muda a `base_url`.

Atencao: a Anthropic **nao tem embeddings**. Nos LAB-002/003/004, use embeddings da OpenAI (`text-embedding-3-small`) ou `sentence-transformers` local.

Passo a passo

1. Abra <https://console.anthropic.com> e crie conta (pede phone verify)
2. Clame o credito free de US\$5 (botao “Claim”) — **so no dia da aula** (expira em 14 dias)
3. Va em **API Keys > Create Key**
4. **Copie a chave** (comeca com `sk-ant-...`)

Modelos e pricing (USD por 1M tokens, conferir antes da aula)

Modelo	Papel	Input	Output
<code>claude-haiku-4-5</code>	cheap (default)	1.00	5.00
<code>claude-sonnet-4-6</code>	premium (LAB-005)	3.00	15.00

Adicionar a chave no `.env`

```
echo 'ANTHROPIC_API_KEY=sk-ant-...sua-chave-aqui' > .env
```

No client (1a celula do notebook), use a base URL da Anthropic:

```
client = OpenAI(
    api_key=os.environ["ANTHROPIC_API_KEY"],
    base_url="https://api.anthropic.com/v1/",
)
MODEL = "claude-haiku-4-5"
```

Testar a chave (qualquer caminho)

Da raiz da pasta (`software-com-ia-generativa/`), com o `.venv` ja sincronizado:

```

uv run python -c "
import os
from dotenv import load_dotenv
from openai import OpenAI
load_dotenv()

def ping(client, model, label):
    r = client.chat.completions.create(
        model=model, messages=[{'role':'user','content':'Diga apenas: ping'}])
    print(f'{label} OK:', r.choices[0].message.content)

if os.getenv('GEMINI_API_KEY'):
    ping(OpenAI(api_key=os.getenv['GEMINI_API_KEY'],
        base_url='https://generativelanguage.googleapis.com/v1beta/openai/'),
        'gemini-2.5-flash-lite', 'Gemini')
elif os.getenv('GROQ_API_KEY'):
    ping(OpenAI(api_key=os.getenv['GROQ_API_KEY'],
        base_url='https://api.groq.com/openai/v1'),
        'llama-3.3-70b-versatile', 'Groq')
elif os.getenv('ANTHROPIC_API_KEY'):
    ping(OpenAI(api_key=os.getenv['ANTHROPIC_API_KEY'],
        base_url='https://api.anthropic.com/v1/'),
        'claude-haiku-4-5', 'Anthropic')
elif os.getenv('OPENAI_API_KEY') and not os.getenv('OPENAI_BASE_URL'):
    ping(OpenAI(), 'gpt-4o-mini', 'OpenAI')
else:
    ping(OpenAI(api_key='ollama-local', base_url='http://localhost:11434/v1'),
        'qwen3:8b', 'Ollama')
"

```

Deve imprimir algo como `Gemini OK: ping`. Se der erro, vá pra [STroubleshooting](#).

Seguranca — 4 regras nao-negociaveis

1. **Nunca** colocar a chave dentro de código Python. So no `.env`.
2. **Nunca** commitar o `.env` (já está no `.gitignore` do repo; o pre-commit hook `gitleaks` bloqueia se você tentar).
3. **Nunca** colar a chave no chat do curso, em Slack público, ou em screenshots.
4. **Revogue a chave ao fim do curso** — login no console do provider e delete.

*Se você acha que vazou a chave (por exemplo, commitou sem querer e fez push), a ação certa é: ir no console do provider, **revogar imediatamente** e gerar uma nova. Reescrever o histórico do git só resolve em parte; assumir que ela está comprometida é mais seguro.*

Troubleshooting

Sintoma	Caminho	Diagnostico	Fix
401 Unauthorized	A / B / D / E	chave invalida ou copiada com espaco	abrir o <code>.env</code> , conferir; gerar nova no console se necessario
429 Too Many Requests	A / B / D / E	rate limit ou credito esgotado	aguardar 60s; ou alternar pra outro caminho
403 Permission denied	A	conta sem acesso a Gemini API	confirmar em <code>aistudio.google.com</code> que voce esta logado na conta certa
insufficient_quota	D / E	credito free de US\$5 expirou (14 dias) ou acabou	usar Caminho A (Gemini free) ou C (Ollama)
ConnectionRefusedError em localhost:11434	C	Ollama nao esta rodando	abrir terminal novo, rodar <code>ollama serve</code>
model 'qwen3:8b' not found	C	modelo nao baixado	rodar <code>ollama pull qwen3:8b</code>
Connection timeout	A / B / D / E	bloqueio de rede corporativa	conectar em outra rede (4G do celular funciona)
KeyError: 'GEMINI_API_KEY' no notebook	qualquer	a 1a celula espera Gemini mas voce usou outro caminho	editar a 1a celula conforme a tabela em “Como o notebook escolhe o provider”

Se nenhum dos itens acima resolveu, peca ajuda no plantao da tarde. Mande print do **erro completo** (nao so a ultima linha) e diga qual caminho voce tentou.

Checklist final antes da aula

- Escolhi **um** caminho (A-E)
 - Chave gerada e copiada no `.env` da raiz da pasta
 - `uv sync` rodou sem erro (ver `SETUP.pdf`)
 - Comando de teste imprimiu `... OK: ping` sem erro
 - Se NAO for o Caminho A, ajustei a 1a celula do notebook conforme a tabela
 - Confirmei que `.env` esta no `.gitignore` (rodar `git status -- .env` **nao** deve aparecer)
 - Salvei a chave fora do repo tambem (gerenciador de senhas), pra nao perder
- Pronto. Voce ja pode rodar `uv run jupyter lab notebooks/` e abrir o notebook `00-cliente-llm-robusto.ipynb`.